

MOD-03 Technical Module Documentation

Acoustics & Navigation Module

Course CSE396 – Computer Engineering Project
Group 7
Project Autonomous First Responder Fire & Rescue Robot
Term Spring 2025–2026
Instructor Salih Sarp

Name	Student ID	Role in MOD-03
Uğur Anıl Güney	210104004011	Primary — STM32 Firmware & Acoustic Processing
Evrin Doğa Solmaz	230104004042	Secondary — Python Bridge & Navigation Interfacing
Tuana Melisa Aksoy	230104004903	Secondary — FSM Branching & Mode Transitions
Dicle Çoban	220104004088	Secondary — Unity Visualizer & Beam Mapping

Submission Date: April 2026

Contents

1	Module Overview	3
2	Purpose of the Module	3
3	Internal Responsibilities of MOD-03	3
3.1	STM32 Acoustic Signal Processing (acoustics_iir.h)	3
3.2	FSM Acoustic Branching (fsm_acoustic.h)	4
3.3	Python Navigation Bridge (acoustic_homing.py)	5
3.4	Unity Acoustic Beam Visualization (MapManager_AcousticBeam.cs)	6
4	Data Structures Used in MOD-03	6
4.1	C / STM32 Layer	6
4.2	Python / Raspberry Pi Layer	7
4.3	C# / Unity Layer	7
5	Connections Between MOD-03 and Other Modules	7
5.1	Connection with MOD-01 (Embedded & Hardware)	7
5.2	Connection with MOD-04 (Web Dashboard & STT)	8
5.3	Connection with MOD-05 (Unity Digital Twin)	8
6	Functional Workflow of MOD-03	8
7	Importance of MOD-03 in the System	9
8	Risks and Design Considerations	9
9	Conclusion	10

1 Module Overview

MOD-03 (Acoustics & Navigation) is the acoustic sensing and autonomous movement layer of the rescue robot. Its responsibility is to detect human distress calls using a three-microphone array, filter out continuous motor noise from the raw audio signal, compute the direction of the sound source, and translate that direction into motor navigation commands that steer the robot toward the victim.

MOD-03 spans three platforms and three programming languages: an STM32 Blue Pill microcontroller running C firmware for signal processing (`acoustics_iir.h`), a shared C header defining the FSM branching logic used by both the STM32 side and the Pi-side main FSM (`fsm_acoustic.h`), a Raspberry Pi 5 running a Python navigation bridge (`acoustic_homing.py`), and a Unity C# component for operator-facing visualization (`MapManager_AcousticBeam.cs`).

2 Purpose of the Module

The purpose of MOD-03 is to provide the robot with directional hearing capability and to connect acoustic detections to autonomous navigation decisions. The system's vision pipeline (MOD-02) requires a direct line of sight to detect a victim. Acoustic localization provides a complementary detection modality that does not depend on visibility, which is important in smoke-filled or obstructed disaster environments.

The core engineering challenge is that the robot's own DC motors and gearboxes generate continuous mechanical noise that masks incoming distress calls on the microphone array. MOD-03 solves this by running a Software-based Digital IIR Filter directly on the STM32, which attenuates motor and gearbox noise from the ADC samples before bearing computation is attempted. This is described in the project proposal as a deliberate choice over static hardware (RC) filters: unlike hardware filters, a software IIR filter can be tuned without physical changes to the circuit.

In addition to acoustic sensing, MOD-03 is also responsible for the initial area mapping behavior. At startup, before exploration begins, the robot executes a 360-degree Spin-Scan that populates a 2D occupancy grid using the four ultrasonic sensors. This grid is the spatial foundation on which the exploration FSM operates.

3 Internal Responsibilities of MOD-03

MOD-03 has four internal subsystems, each defined in a separate file and each operating on a different platform.

3.1 STM32 Acoustic Signal Processing (`acoustics_iir.h`)

This subsystem runs on the STM32 Blue Pill microcontroller. It owns the lowest-level acoustic signal pipeline, from raw ADC samples to a computed bearing angle and occupancy grid. The subsystem exposes four public functions:

- `acoustics_iir_filter_apply()` applies a 4th-order Software-based Digital IIR filter (`ACOUSTICS_IIR_ORDER = 4`) to the raw ADC sample buffer from each microphone. Its

purpose is to attenuate motor and gearbox noise before any bearing computation is attempted.

- `acoustics_compute_bearing()` receives the filtered sample buffers from all three MAX4466 microphones and computes the bearing angle to the acoustic source via phase difference across the microphone array. The result is stored in an `acoustics_result_t` struct containing the bearing in degrees, a `hit_detected` flag, and a timestamp. A detection is only flagged when the computed confidence meets or exceeds `ACOUSTICS_HIT_THRESHOLD = 0.6f`.
- `acoustics_spinscan_execute()` drives the robot through a complete 360-degree rotation while pinging all four HC-SR04 ultrasonic sensors at each step. The collected distance readings populate an `acoustics_grid_t` occupancy grid of dimension `ACOUSTICS_GRID_SIZE = 64` cells per side. This function is called once during the `FSM_STATE_SPIN_MAP` startup phase.
- `acoustics_homing_navigate()` converts a confirmed bearing angle into a low-level motor direction command using the `acoustics_nav_cmd_t` enum, which is aligned with MOD-01's `uart_direction_t`: 0=STOP, 1=FORWARD, 2=BACKWARD, 3=LEFT, 4=RIGHT.

3.2 FSM Acoustic Branching (`fsm_acoustic.h`)

This subsystem defines the finite state machine branching logic that governs transitions triggered by acoustic detections. It is authored for use by both the STM32 side and, mirrored 1:1, by the Pi-side main FSM (`M4_MainFSM.py`). The header defines all 11 FSM states shared across the project and the decision logic for acoustic-triggered state transitions.

The following constants govern acoustic FSM behavior:

- `FSM_ACOUSTIC_MIN_CONFIRMS = 3` — the robot must receive at least three consecutive `A_Hit = 1` readings before committing to an `EXPLORE → ACOUSTIC_HOMING` transition. This guards against false positives from brief noise events.
- `FSM_ACOUSTIC_HOMING_TIMEOUT_MS = 30000` — if the robot remains in `ACOUSTIC_HOMING` for more than 30 seconds without reaching a victim, the FSM automatically falls back to `EXPLORE`.
- `FSM_ACOUSTIC_COOLDOWN_MS = 3000` — a mandatory 3-second cooldown between two consecutive homing transitions prevents rapid oscillation caused by acoustic echoes or reflections immediately after a homing session ends.
- `FSM_BEARING_DEAD_ZONE_DEG = 10.0f` — bearings within $\pm 10^\circ$ of zero are treated as “source directly ahead,” and the robot drives forward rather than turning.

The subsystem exposes six public functions:

- `FSM_Acoustic_Init()` resets the internal hit streak counter, cooldown timer, and homing session timestamp. Must be called once at startup before the main FSM loop begins.
- `FSM_Acoustic_Update()` is the core branching function called on every FSM tick. It accumulates `A_Hit` confirmations and, once `FSM_ACOUSTIC_MIN_CONFIRMS` consecutive hits are confirmed and the cooldown has elapsed, fires the `EXPLORE → ACOUSTIC_HOMING` transition by populating the `fsm_acoustic_result_t` output struct.

- `FSM_Acoustic_IsHomingTimedOut()` checks whether the current homing session has exceeded `FSM_ACOUSTIC_HOMING_TIMEOUT_MS`. Returns `true` when the FSM should fall back to `EXPLORE`.
- `FSM_Acoustic_ShouldInterruptExplore()` is a lightweight predicate that allows the Python navigation bridge to pre-emptively stop motors before `FSM_Acoustic_Update()` formally changes the FSM state.
- `FSM_Acoustic_ResetStreak()` clears the hit counter and cooldown state. Called on homing timeout or false positive identification.
- `FSM_Acoustic_GetStateName()` returns a human-readable string label for any `fsm_state_t` value. Used by MOD-04's WebSocket telemetry and MOD-05's Unity Digital Twin for status display and logging.

3.3 Python Navigation Bridge (`acoustic_homing.py`)

This subsystem runs on the Raspberry Pi 5 and acts as the high-level bridge between incoming acoustic telemetry and the robot navigation system. Unlike the C headers above, which define interfaces, `acoustic_homing.py` contains a fully implemented concrete class, `AcousticHomingBridge`, that implements the `IAcousticHomingBridge` abstract interface.

`AcousticHomingBridge` exposes three public methods:

- `process_telemetry()` is called on every FSM tick with a parsed `AcousticTelemetry` object. It mirrors the decision logic of `FSM_Acoustic_Update()` in Python: if `a_hit` is `False`, the hit streak is reset and `None` is returned. If `a_hit` is `True`, the streak increments. Once the streak reaches `HOMING_MIN_HIT_CONFIRMS = 3` and the robot is not yet in homing mode, it calls `notify_fsm_transition()` and enters homing mode. On each subsequent call while homing, it produces a `NavCommand` by converting the current bearing angle via the internal `_bearing_to_nav_command()` helper. If `HOMING_TIMEOUT_SEC = 30.0` seconds elapse, `reset()` is called automatically and a `STOP` command is returned.
- `notify_fsm_transition()` notifies the MOD-04 main FSM to transition from `EXPLORE` to `ACOUSTIC_HOMING`. If a callback was provided at construction it is invoked; otherwise the event is only logged.
- `reset()` clears the hit streak counter, homing flag, and start timestamp. Called by MOD-04 when the homing session times out or a false positive is identified, mirroring `FSM_Acoustic_ResetStreak()` on the C side.

The internal `_bearing_to_nav_command()` helper applies the following mapping, aligned with `FSM_BEARING_DEAD_ZONE_DEG = 10.0`:

- $|bearing| \leq 10^\circ \rightarrow$ FORWARD at `HOMING_APPROACH_SPEED = 100` PWM, with `buzzer_on = True` and `lights_on = True`.
- $bearing > 10^\circ$ (source to the right) $\rightarrow$ RIGHT at `HOMING_DEFAULT_SPEED = 150` PWM, with `lights_on = True`.
- $bearing < -10^\circ$ (source to the left) $\rightarrow$ LEFT at `HOMING_DEFAULT_SPEED = 150` PWM, with `lights_on = True`.

3.4 Unity Acoustic Beam Visualization (`MapManager_AcousticBeam.cs`)

This subsystem is the only MOD-03 component that runs inside the Unity environment. It is a `MonoBehaviour` that attaches alongside `MapManager.cs` on the `MapManager` `GameObject` and is responsible for rendering the detected acoustic bearing as a directional indicator on the operator's 2D top-down map.

The component exposes three public methods:

- `ShowAcousticBeam()` renders the bearing as a visual indicator on the Unity map. It accepts an `AcousticBeamData` object (containing bearing, robot position, and timestamp) and an `AcousticBeamStyle` enum value. The visual is implemented using a `Unity LineRenderer` component. If `hitDetected` is `false`, it immediately delegates to `HideAcousticBeam()`.
- `HideAcousticBeam()` deactivates the beam `GameObject`. Called by `RobotManager` when the FSM exits `ACOUSTIC_HOMING` and `TelemetryData.acousticHit` returns to `false`.
- `UpdateAcousticBeamAngle()` updates the bearing of an already-visible beam without recreating the object. Used when consecutive readings refine the bearing estimate during an active homing session.

Two visual styles are supported via the `AcousticBeamStyle` enum: `DirectionArrow` renders a static cyan arrow of length `BEAM_ARROW_LENGTH = 1.5f` Unity units; `RadarSweep` animates the arrow using `Mathf.Sin` in the component's `Update()` loop over a sweep duration of `BEAM_SWEEP_DURATION = 0.8f` seconds.

The internal `GridToWorldPosition()` helper converts the robot's grid-space coordinates from `TelemetryData` into Unity world-space coordinates, using the same grid-to-world conversion pattern as `MapManager.cs`.

4 Data Structures Used in MOD-03

MOD-03 defines data structures across three platforms. All structures are named and documented as they appear in the source files.

4.1 C / STM32 Layer

`acoustics_status_t` is the return code enum used by all STM32 acoustic functions. Values: `ACOUSTICS_OK = 0`, `ACOUSTICS_ERR_INIT = -1`, `ACOUSTICS_ERR_TIMEOUT = -2`, `ACOUSTICS_ERR_NO_HIT = -3`.

`acoustics_result_t` stores the output of one bearing computation cycle. Fields: `bearing_deg` (float, -180 to $+180$ degrees), `hit_detected` (bool), `timestamp_ms` (uint32, milliseconds since boot).

`acoustics_grid_t` is the 2D occupancy grid populated by the Spin-Scan. Fields: `cells[64][64]` (uint8 array where 0 = free, 1 = obstacle), `size` (uint8, grid dimension per side), `resolution_cm` (float, real-world size of each cell in centimetres).

`acoustics_nav_cmd_t` is the motor direction output enum from `acoustics_homing_navigate()`, aligned with MOD-01's `uart_direction_t`: `STOP=0`, `FORWARD=1`, `BACKWARD=2`, `LEFT=3`,

RIGHT=4.

`fsm_state_t` defines all 11 top-level FSM states shared across the project. Integer values are the wire format used by MOD-05 Unity as well: IDLE=0, SPIN_MAP=1, EXPLORE=2, ACOUSTIC_HOMING=3, APPROACH_TARGET=4, VICTIM_ANALYSIS=5, EVALUATE_VICTIM=6, WAKEUP_PROTOCOL=7, MANUAL_OVERRIDE=8, BEACON_MODE=9, RTH=10.

`fsm_acoustic_event_t` is the acoustic event payload passed to `FSM_Acoustic_Update()` after being parsed from the UART telemetry string. Fields: `a_hit` (bool, from `acoustics_result_t.hit_detected`), `a_ang` (float, from `uart_telemetry_t.acoustic_angle`), `timestamp_ms` (uint32).

`fsm_acoustic_result_t` is the output of one `FSM_Acoustic_Update()` call. Fields: `new_state` (`fsm_state_t`), `transition_fired` (bool, true if a state transition was committed), `confirmed_bearing` (float, valid only when `transition_fired` is true).

4.2 Python / Raspberry Pi Layer

`AcousticTelemetry` is a Python dataclass parsed from the MOD-01 UART telemetry string by MOD-04's serial reader. Fields: `a_hit` (bool), `a_ang` (float, -180.0 to $+180.0$ degrees).

`MotorDirection` is a Python integer enum aligned with MOD-01's `uart_direction_t`. Values: STOP=0, FORWARD=1, BACKWARD=2, LEFT=3, RIGHT=4.

`NavCommand` is a Python dataclass representing the motor instruction produced by the homing bridge and forwarded to MOD-01 via UART. Fields: `direction` (`MotorDirection`), `speed` (int, 0–255 PWM value), `buzzer_on` (bool, default False), `lights_on` (bool, default False).

4.3 C# / Unity Layer

`AcousticBeamStyle` is a C# enum controlling the visual rendering mode of the beam indicator. Values: `DirectionArrow` = 0, `RadarSweep` = 1.

`AcousticBeamData` is a serializable C# class populated from `TelemetryData` by `RobotManager` when `acousticHit` is true. Fields: `bearingDeg` (float), `hitDetected` (bool), `posX` (float), `posY` (float), `timestampMs` (uint).

5 Connections Between MOD-03 and Other Modules

5.1 Connection with MOD-01 (Embedded & Hardware)

MOD-03 has the most direct hardware-level coupling with MOD-01. On the STM32, the acoustic processing functions in `acoustics_iir.h` produce an `acoustics_result_t`. The STM32 firmware appends this result to MOD-01's `uart_telemetry_t` struct via the `acoustic_angle` field, transmitted to the Raspberry Pi at 50 Hz as part of the UART telemetry wire format:

```
T:<temp>|S:<smoke>|ST:<stuck>|A_Ang:<angle>|Y:<yaw>|UF:<front>|...
```

In the reverse direction, the `NavCommand` produced by `AcousticHomingBridge` is serialized and sent to the STM32 as a `uart_command_t` struct, which carries the direction integer (0–4), speed PWM value (0–255), `buzzer_on`, and `lights_on` flags. This command is

consumed directly by MOD-01's `motor_set_state()` and the buzzer/LED GPIO control.

5.2 Connection with MOD-04 (Web Dashboard & STT)

The FSM coordination connection. MOD-04's main FSM loop (`M4_MainFSM.py`) mirrors the logic of `FSM_Acoustic_Update()` exactly. As stated directly in the `fsm_acoustic.h` header, the Python function `fsm_acoustic_update()` is a 1:1 mapping of the C function. Both layers therefore enforce the same 3-confirmation requirement, the same 30-second timeout, and the same 3-second cooldown.

MOD-04 is also the consumer of the transition callback registered in `AcousticHomingBridge`. When `notify_fsm_transition()` fires, it invokes the callback provided by MOD-04, which formally transitions the main FSM state from `EXPLORE` to `ACOUSTIC_HOMING`. When the homing session ends (timeout or false positive), MOD-04 calls `AcousticHomingBridge.reset()`, mirroring `FSM_Acoustic_ResetStreak()` on the C side.

Additionally, MOD-04 packages `acoustic_hit` and `acoustic_angle` into the `AugmentedStatusReport` that is broadcast to the Unity dashboard via `WebSocket`.

5.3 Connection with MOD-05 (Unity Digital Twin)

The connection between MOD-03 and MOD-05 is indirect but visually significant. MOD-04 sends the `AugmentedStatusReport` JSON to Unity via `WebSocket`. The `UnityRobotManager.HandleTel` method reads `TelemetryData.acousticHit`: if `true`, it constructs an `AcousticBeamData` object from the telemetry fields and calls `MapManager_AcousticBeam.ShowAcousticBeam()`; if `false`, it calls `HideAcousticBeam()`.

It is worth noting that `MapManager_AcousticBeam.cs` is physically located in the `MOD-03_Acoustics` folder of the repository, as it defines a visualization contract that belongs to the acoustic subsystem, but executes inside MOD-05's Unity scene at runtime.

6 Functional Workflow of MOD-03

A complete acoustic detection cycle, from raw microphone input to Unity visualization, proceeds as follows:

1. The STM32 ADC continuously samples the three MAX4466 microphones at `ACOUSTICS_SAMPLE_RATE = 8000 Hz`.
2. `acoustics_iir_filter_apply()` is called on each microphone's sample buffer to remove motor and gearbox noise from the signal.
3. `acoustics_compute_bearing()` processes the filtered buffers and computes the bearing to the sound source via phase difference. If the computed confidence is ≥ 0.6 , `hit_detected` is set to `true` and the bearing angle is stored in `acoustics_result_t`.
4. The STM32 packs the bearing into `uart_telemetry_t.acoustic_angle` and transmits the UART telemetry string to the Raspberry Pi at 50 Hz.
5. MOD-04 on the Pi parses the incoming UART string, extracts `A_Hit` and `A_Ang`, and creates an `AcousticTelemetry` object.

6. `AcousticHomingBridge.process_telemetry()` is called with the parsed telemetry. If `a_hit` is `False`, the hit streak is reset and `None` is returned.
7. If `a_hit` is `True`, the internal streak counter increments. While the streak is below `HOMING_MIN_HIT_CONFIRMS = 3`, no action is taken and `None` is returned.
8. When the streak reaches 3, `notify_fsm_transition()` fires. The MOD-04 FSM transitions from `EXPLORE` to `ACOUSTIC_HOMING`.
9. `_bearing_to_nav_command()` converts the current bearing into a `NavCommand`. This `NavCommand` is forwarded via UART to the STM32 as a `uart_command_t`, which drives the motors to rotate toward the sound source.
10. MOD-04 includes `acoustic_hit = True` and `acoustic_angle = <bearing>` in the `AugmentedStatusReport` and broadcasts it to Unity via `WebSocket`.
11. `Unity RobotManager.HandleTelemetryReceived()` detects `data.acousticHit == true` and calls `MapManager_AcousticBeam.ShowAcousticBeam()`, rendering a directional arrow on the operator's 2D map.
12. If 30 seconds elapse without finding the source, `FSM_Acoustic_IsHomingTimedOut()` returns `true`, MOD-04 calls `AcousticHomingBridge.reset()`, the FSM returns to `EXPLORE`, and `HideAcousticBeam()` is called in Unity.

7 Importance of MOD-03 in the System

MOD-03 serves two distinct roles that no other module covers.

First, it provides the acoustic detection modality. The overall system design is described in the project proposal as Tri-Modal Sensor Fusion: Vision (MOD-02), Acoustic (MOD-03), and Environmental (MOD-01 sensors). Removing MOD-03 reduces the system to two modalities and eliminates the robot's ability to locate victims who cannot be seen by the camera.

Second, MOD-03 is responsible for the initial Spin-Scan that builds the 2D occupancy grid. Without this grid, the exploration FSM has no spatial map to work from when the robot enters `EXPLORE` mode. This makes MOD-03 a prerequisite for the broader autonomous navigation behavior, not only for acoustic homing.

For the operator, the acoustic beam visualization in `MapManager_AcousticBeam.cs` provides real-time situational awareness: the operator can see exactly which direction the robot is turning toward, which is important for making informed manual override decisions during a rescue mission.

8 Risks and Design Considerations

The main technical challenges in MOD-03 and the design decisions taken to address them are summarized below.

Motor and gearbox noise. The robot's own motors continuously generate acoustic noise on the microphone array. This is mitigated by `acoustics_iir_filter_apply()`, which attenuates motor and gearbox noise from the ADC samples before bearing computation

is attempted.

Echo and reflection in indoor arenas. Indoor environments produce acoustic reflections that can cause a single distress call to appear as multiple hits arriving from slightly different angles. This is mitigated by requiring `FSM_ACOUSTIC_MIN_CONFIRMS = 3` consecutive `A_Hit` readings before committing to a state transition. A single echo is unlikely to produce three consecutive confirmations.

Rapid re-triggering after a homing session. Echoes or residual signal energy immediately after a completed homing session could trigger a new `ACOUSTIC_HOMING` transition. This is mitigated by `FSM_ACOUSTIC_COOLDOWN_MS = 3000`, a mandatory 3-second cooldown between two consecutive homing transitions.

Chasing phantom sounds indefinitely. If the robot enters `ACOUSTIC_HOMING` toward a false sound source, it must be able to recover. This is mitigated by `FSM_ACOUSTIC_HOMING_TIMEOUT_MS = 30000`: after 30 seconds in homing mode without success, the FSM automatically falls back to `EXPLORE` and `reset()` is called.

Open question. As noted in the MOD-03 README, it has not yet been decided whether `acoustics_spinscan_execute()` should run only once at startup (`SPIN_MAP` state) or be re-triggered periodically if no victims are found within a set time during exploration. This decision affects both the navigation strategy and the freshness of the occupancy grid.

9 Conclusion

MOD-03 is the acoustic sensing, FSM branching, and navigation layer of the rescue robot. Its design is divided into four separate subsystems: the STM32 signal processing layer handles raw audio filtering and bearing computation at the hardware level; the FSM branching header defines the confirmation and transition logic shared between the embedded and Pi-side FSMs; the Python navigation bridge implements the concrete homing behavior and translates acoustic events into motor commands; and the Unity visualization component renders the bearing information for the operator in real time.

Through its connection with MOD-01 via UART telemetry and motor commands, its FSM coordination with MOD-04, and its visualization role in MOD-05, MOD-03 is integrated across the full depth of the system stack. Together with MOD-02 (Vision) and the environmental sensors of MOD-01, it forms one of the three detection modalities of the Tri-Modal Sensor Fusion architecture described in the project proposal.